A Collaborative Computation Offloading Framework based on Multi-Agent Deep Reinforcement Learning in Partially Observable Mobile Edge Computing

Iman Rahmaty , Hamed Shah-Mansouri , Mohammad Hosseini , and Ali Movaghar

***Abstract*—Mobile edge computing (MEC) is a promising computing scheme to support computation-intensive AI applications in edge networks. We study collaborative computation offloading in MEC to support computation-intensive applications. Mobile devices (MDs) can offload their computation to edge nodes (ENs), where we leverage edge-to-edge offloading to further enhance the MEC's computing capabilities. This however presents significant challenges due to the need for real-time and decentralized decision-making in the highly dynamic MEC environment especially with collaborative offloading. We design a queue-based multi-layer model scenario and formulate the joint offloading problem as a decentralized partially observable markov decision process (Dec-POMDP), where each MD and EN constructs and trains offloading agents to achieve high performance and efficient resource utilization in MEC. To solve the formulated problem, we propose a multi-agent deep reinforcement learning (DRL)-based approach, where multiple agents collaborate to make distributed decisions in an uncertain MEC environment through global optimization.**

***Index Terms*—Mobile edge computing, Computation offloading, Des-POMDP, Multi-agent deep reinforcement learning**

I. Introduction

To meet the rapidly growing demands of computation-intensive and time-sensitive AI applications [1], mobile edge computing (MEC) [2] is emarged as a promissing solution, by allowing mobile devices (MDs) to offload computational tasks to edge computing nodes (ENs) located near to them. However, MEC often suffers from their dynamic and uncertain environment, such as time-varying conditions, heterogeneous devices, and frequent communication requests. Reinforcement learning (RL) [3] has been widely proposed as an effective tool that enables MEC network entities to interact with the environment for learning an optimal policy, usually modeled as a markov decision process (MDP) [4].

In an MEC-enabled network, a key challenge is determining whether and how to offload each task, as well as selecting the appropriate computing entities and transmission links for computation offloading. This problem has been widely investigated in recent RL-based research, where tasks from MDs are offloaded to ENs through device-to-edge (D2E) transmission links. For example, Zhao *et al.* in [5] developed a computation offloading algorithm to address competition for wireless channels. To handle deadline-constrained computational tasks, Tang *et al.* in [6] introduced a distributed offloading algorithm that manages uncertain workload dynamics at the edge nodes. Zhou *et al.* in [7] investigated the joint optimization of computation offloading and resource allocation to minimize energy consumption across the entire MEC. Liao *et al.* in [8] introduced a Deep RL (DRL)-based algorithm for performing online computation offloading in MEC. This algorithm optimizes transmission power and CPU frequency when minimizing both task computation delay and energy consumption. Sun *et al.* in [9] tackled both computation offloading and service caching problems, proposing a hierarchical DRL framework to minimize long-term average service delay. We also in [10] studied the computation offloading problem under strict task processing deadlines and energy constraints, proposing a distributed algorithm to maximize each user's long-term QoE individually. However, research [5]–[10] primarily overlooked the potential for collaboration among ENs, specifically failing to leverage underutilized computing resources that could be shared among multiple ENs to enhance network efficiency. To achieve optimal utilization and high performance in MEC environments, it is worth exploring a collabrative offloading framework, where both MD and EN can offload computation-intensive tasks to a particular EN through D2E and edge-to-edge (E2E) transmission links, respectively, according to their computation and communication capacities.

The problem are challenging due to the online and asynchronous decision-making requirement for each task, while multiple different decision-makers collaborate to asynchronous offloading decision-making. Specifically, each MD and EN should act as different agents to make offloading decisions based on their partial observation of the global network state, which makes single-agent RL methods inefficient and sometimes inapplicable. Single-agent RL learns its decision-making policy independently and treats other agents as part of the environment, which may cause the non-stationarity issue [11] and significantly reduce learning efficiency and network performance. To tackle with the non-stationarity issue in MEC environments, RL-based studies have turned to the use of multi-agent RL (MARL)-based methods (e.g., [12], [13], [14], and [15]), where multiple agents collaborate to make distributed decisions through global optimization.

In this work we are interested in exploring a collaborative offloading framework, while effectively address multi-agent systems challenges. We formulated the collaborative offloading problems for both MDs and ENs as a decentralized partially observable markov decision process (Dec-POMDP) [16], where different decision-makers interact with each other based on partial observations and limited information about the network and other entities to achieve overall system performance. To solve the problem, we propose a multi-agent DRL-based

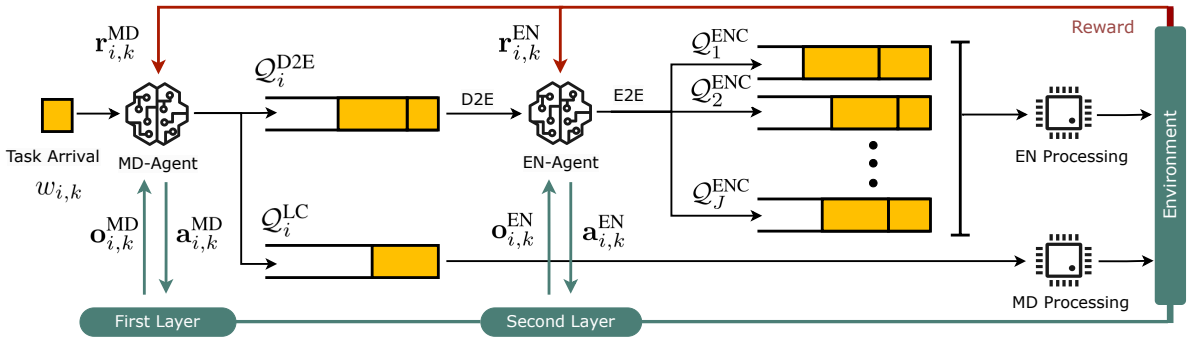


Fig. 1. An illustration of MD and EN agents offloading decision-making process for arrival task in the queue-based collaborative offloading framework.

framework, where each MD and EN constructs and trains its own agent to make different layers' decisions. The key idea of this collaborative framework is to decouple task offloading decisions by making decisions at different layers at different time points. This approach allows each layer's decision-making processing to dynamically adjust based on the current network state and resource availability, thereby improving the overall efficiency and responsiveness of the system. Specifically, we design a multi-agent dueling double deep Q-network (D3QN) model [17], [18], where each agent has a decentralized actor and a centralized critic and can be applied to collaboration in multi-agent settings. Overall, this work aims to provide a novel collabrative framework for computation offloading problem in MEC, addressing the challenges of online decision-making and multi-agent environments. In summary, the key contributions of this work are summarized as follows.

1) **Novel Collabrative Model Scenario:** We consider a novel queue-based collaborative computation offloading framework in an MEC-enabled network, which incorporates both D2E and E2E offloading mechanism, and can significantly enhance the offloading performance.
2) **Novel MARL-Based Solution:** We focus on computation offloading problem, aiming to tackle the online and asynchronous decision-making challenge in multi-agent environments. We propose MARL-based algorithms, which enable MDs and edge ENs to collaboratively make distributed decisions through global optimization.

The remainder of this work is organized as follows. We first present the collaborative offloading system model in Section II, after which we formulate the optimization problem and propose the MARL-based solution in Section III.

II. Queue-based collaborative offloading

We consider a general MEC model that consists a set of MDs $\mathcal{I} = \{1, \ldots, I\}$, and a set of ENs $\mathcal{J} = \{1, \ldots, J\}$. We regard the time as a specific episode containing a series of time slots $\mathcal{T} = \{1, \ldots, T\}$, each with a duration of τ seconds. Each MD can offload its tasks to a local EN via a D2E link, and local EN is able to offload the incoming task to other ENs via E2E links. To accurately characterize task flows across communication links and computing servers, we introduce a First-In-First-Out (FIFO) queueing system and propose a queue-based collaborative offloading framework, as depicted in Fig. 1. Specifically, each MD maintains two FIFO queues, a local computation queue (LC queue), representing the flow of tasks processed locally; and a D2E transmission queue (D2E queue), representing the flow of tasks to be offloaded to ENs via D2E links. Furthermore, each EN maintains a single FIFO queue, referred to as the EN computation queue (ENC queue), which represents the computation flow of all tasks offloaded to the corresponding ENs.

At the beginning of each time slot $t \in \mathcal{T}$, each MD generates new tasks following a Poisson distribution with an expectation of λ. In other words, each MD may generate multiple tasks or no task during a specific time slot. Let K_i denote the total number of tasks generated by MD i, and let $\mathcal{K}_i$ denote the newly arrived task set of MD i in one episode, respectively. That is $\mathcal{K}_i = \{1, 2, \ldots, K_i\}$. For convenience, let $\mathcal{K}_i(t)$ represent the task set generated by MD i at time slot t. Then, we have $\mathcal{K}_i = \bigcup_{t \in \mathcal{T}} \mathcal{K}_i(t)$ to represent the unique index of the k-th task generated by MD i. To characterize the attributes of tasks, we define the notation $w_{i,k} = \{d_{i,k}, c_{i,k}, \tau_{i,k}, \delta_{i,k}\}, k \in \mathcal{K}_i(t)$, where $d_{i,k} \in [d_i^{\min}, d_i^{\max}]$, $c_{i,k} \in [c_i^{\min}, c_i^{\max}]$, $\tau_{i,k} \in \mathcal{T}$, and $\delta_{i,k} \in [\delta_i^{\min}, \delta_i^{\max}]$ denote the data size (in bits), the number of CPU cycles required, the generation time, and the deadline (in seconds) of the task, respectively. We define the binary variable $X_{i,k}^{\text{MD}}$ to represent the MD $i \in \mathcal{I}$ offloading decision for task $w_{i,k}$, where $X_{i,k}^{\text{MD}} = 0$ indicates that the task is assigned to the LC queue, and $X_{i,k}^{\text{MD}} = 1$ denotes assignment to the D2E queue. We also define vector $Y_{i,k,j}^{\text{EN}} = (y_{i,k,j}(t))_{j \in \mathcal{J}}$ to denote the EN $j \in \mathcal{J}$ offloading decision for task $w_{i,k}$, where $y_{i,k,j}(t)$ indicates whether arrived task from MD i to local EN is assigned to EN $j \in \mathcal{J}$'s associated ENC queue ($y_{i,k,j}(t) = 1$) or not ($y_{i,k,j}(t) = 0$).

if MD i chooses to process its task $w_{i,k}$ locally, the task is pushed into the MD i's LC queue for processing. Thus, the total local latency $T_{i,k}^{\text{loc}}$ for the task consists of the local waiting time $T_{i,k}^{\text{loc-w}}$ and the local computation time $T_{i,k}^{\text{loc-c}}$, i.e.,

$$T_{i,k}^{\text{loc}} = T_{i,k}^{\text{loc-w}} + T_{i,k}^{\text{loc-c}}. \tag{1}$$

When task $w_{i,k}$ of MD i is pushed into the LC queue at time slot $\tau_{i,k}$, it observes an ordered task set $\mathcal{Q}_i^{\text{LC}}(\tau_{i,k}) = \{k_1, k_2, \ldots, k_{N_{i,k}}\}$, where $N_{i,k}$ denotes the number of tasks in the set, and each element $k' \in \mathcal{Q}_i^{\text{LC}}(\tau_{i,k})$ represents a task

$w_{i,k'}$ that has already been pushed into the LC queue before the current time slot $\tau_{i,k}(\geq \tau_{i,k'})$. The task needs to wait for local processing until all previous tasks in the LC queue have been completed. It is important to note that a specific task $w_{i,\hat{k}}^{\text{LC}}$ in the queue may not be fully completed. We denote $\phi_{i,\hat{k}}^{\text{LC}} \in [0,1]$ as the percentage of remaining uncompleted workload of the $\hat{k}$-th task in $\mathcal{Q}_i^{\text{LC}}(\tau_{i,k})$. Thus, the local waiting time $T_{i,k}^{\text{loc-w}}$ and the local computation time can be expressed as

$$T_{i,k}^{\text{loc-w}} = \sum_{k' \in \mathcal{Q}_i^{\text{LC}}(\tau_{i,k})} \frac{c_{i,k'}}{f_i^{\text{LC}}} + \frac{\phi_{i,\hat{k}}^{\text{LC}} c_{i,\hat{k}}^{\text{LC}}}{f_i^{\text{LC}}}, \tag{2}$$

$$T_{i,k}^{\text{loc-c}} = \frac{c_{i,k}}{f_i^{\text{LC}}}. \tag{3}$$

On the other hand if MD i chooses to offload its task $w_{i,k}$ to local EN via D2E link, the task is pushed into the D2E queue of the MD for data transmission. Furthermore, if the task in the D2E queue is sent to local EN, it will be scheduled to an EN $j \in \mathcal{J}$ based on local EN's offloading desicion and will be pushed into the ENC queue of EN j and wait for EN's processing. Thus, the total latency $T_{i,k}^{\text{D2E}}$ for the task contains the D2E waiting time $T_{i,k}^{\text{D2E-w}}$, the D2E transmission time $T_{i,k}^{\text{D2E-t}}$, the EN waiting time $T_{i,k}^{\text{EN-w}}$, and the EN computation time $T_{i,k}^{\text{EN-c}}$, i.e.,

$$T_{i,k}^{\text{D2E}} = T_{i,k}^{\text{D2E-w}} + T_{i,k}^{\text{D2E-t}} + T_{i,k}^{\text{EN-w}} + T_{i,k}^{\text{EN-c}}. \tag{4}$$

It is worth noting that the transmission time between ENs is not considered, as the E2E links are assumed to be high-capacity and low-latency connections, making their impact negligible compared to D2E and EN delays.

When a task $w_{i,k}$ of MD i is pushed into the D2E queue at time slot $\tau_{i,k}$, it observes an ordered task set $\mathcal{Q}_i^{\text{D2E}}(\tau_{i,k})$. The task must wait for data transmission until all previous tasks in the D2E queue have been fully transmitted. We denote $\phi_{i,\hat{k}}^{\text{D2E}} \in [0,1]$ as the percentage of remaining untransmitted workload of the $\hat{k}$-th task in the queue $\mathcal{Q}_i^{\text{D2E}}(\tau_{i,k})$. Accordingly, the D2E waiting time $T_{i,k}^{\text{D2E-w}}$ and the D2E transmission time $T_{i,k}^{\text{D2E-t}}$ of task $w_{i,k}$ are given by

$$T_{i,k}^{\text{D2E-w}} = \sum_{k' \in \mathcal{Q}_i^{\text{D2E}}(\tau_{i,k})} \frac{d_{i,k'}}{r_{i,k'}^{\text{D2E}}} + \frac{\phi_{i,\hat{k}}^{\text{D2E}} d_{i,\hat{k}}^{\text{D2E}}}{r_{i,\hat{k}}^{\text{D2E}}}, \tag{5}$$

$$T_{i,k}^{\text{D2E-t}} = \frac{d_{i,k}}{r_{i,k}^{\text{D2E}}}. \tag{6}$$

At time $t_*^{\text{D2E}} = \tau_{i,k} + T_{i,k}^{\text{D2E-w}} + T_{i,k}^{\text{D2E-t}}$, task $w_{i,k}$ arrives at EN j, which is selected by the local EN agent ($y_{i,k,j} = 1$). Upon arrival at EN j, the task $w_{i,k}$ observes the ordered task set $\mathcal{Q}_j^{\text{ENC}}(t_*^{\text{D2E}})$, whose element $(i',k') \in \mathcal{Q}_j^{\text{ENC}}(t_*^{\text{D2E}})$ represents task $w_{i',k'}$ that has been pushed into the ENC queue of EN j before the time of $t_1^{\text{D2E}}(\geq \tau_{i,k})$. It is important to note that a specific task $w_{\hat{i},\hat{k}}^{\text{ENC}}$ may not be completed. we denote $\phi_{\hat{i},\hat{k}}^{\text{EN}} \in [0,1]$ as the percentage of remaining uncompleted workload of the $\hat{k}$th task of MD $\hat{i}$ in the queue $\mathcal{Q}_j^{\text{ENC}}(t_*^{\text{D2E}})$. Thus, the ENC waiting time $T_{i,k}^{\text{EN-w}}$ and EN computation time $T_{i,k}^{\text{EN-c}}$ of task $w_{i,k}$ can be calculated in the following:

$$T_{i,k}^{\text{EN-w}} = \sum_{j \in \mathcal{J}} y_{i,k,j} \sum_{(i',k') \in Q_j^{\text{ENC}}(t_*^{\text{D2E}})} \frac{c_{i',k'}}{f_j^{\text{EN}}} + \frac{\hat{\phi}_{i,\hat{k}}^{\text{EN}} c_{i,\hat{k}}^{\text{EN}}}{f_j^{\text{EN}}}, \tag{7}$$

$$T_{i,k}^{\text{EN-c}} = \sum_{j \in \mathcal{J}} y_{i,k,j} \frac{c_{i,k}}{f_j^{\text{EN}}}.$$

III. MDP-Based Problem Formulation

Based on system model, we formulated the collaborative offloading problem as a Dec-POMDP [16], where each MD and EN acts as an autonomous agent to learn an optimal collaborative offloading strategy through continuous interaction with the environment. Specifically, a Dec-POMDP is characterized by the state space $\mathcal{S}$, the action space $\mathcal{A}$, and the observation space $\mathcal{O}$ for each agent. At each decision epoch $\tau_{i,k}$ corresponding to the arrival of task $w_{i,k}$, MD i and EN j receive their local observation $\mathbf{o}_{i,k}^{\text{MD}} \in \mathcal{O}$ and $\mathbf{o}_{i,k}^{\text{EN}} \in \mathcal{O}$, respectively, which are a part of the global state $\mathcal{S}$, i.e.,

$$\mathbf{o}_{i,k}^{\text{MD}} = \Big(w_{i,k}, Q_i^{\text{LC}}(\tau_{i,k}), Q_i^{\text{D2E}}(\tau_{i,k})\Big), \tag{8}$$

$$\mathbf{o}_{i,k}^{\text{EN}} = \Big(w_{i,k}, Q_j^{\text{ENC}}(\tau_{i,k})\Big), \quad j \in \mathcal{J}, \tag{9}$$

where MD's observation $\mathbf{o}_{i,k}^{\text{MD}}$ is limitted to its local queues information, while EN's observation consists all ENs' queue information at time $\tau_{i,k}$. Upon observing the current offloading state, each MD or EN agent selects an offloading action for task $w_{i,k}$:

$$\mathbf{a}_{i,k}^{\text{MD}} \triangleq X_{i,k}^{\text{MD}}, \tag{10}$$

$$\mathbf{a}_{i,k}^{\text{EN}} \triangleq Y_{i,k,j}^{\text{EN}}, \ \ j \in \mathcal{J}. \tag{11}$$

Then, each agent receives a reward $\mathbf{r}_{i,k}$, which depends on the actions of both the MD and EN agents in the case of D2E offloading. Accordingly, the reward associated with task $w_{i,k}$ is attributed to the pair of agents that jointly made the offloading decision for this task, based on the total completion delay experienced by it, i.e., $\mathbf{r}_{i,k}^{\text{EN}} = \mathbf{r}_{i,k}^{\text{MD}} =$

$$\mathbf{r}_{i,k} = -\Big(X_{i,k}^{\text{MD}} T_{i,k}^{\text{loc}} + \big(1 - X_{i,k}^{\text{MD}}\big) \sum_{j \in \mathcal{J}} y_{i,k,j}^{\text{EN}} T_{i,k}^{\text{D2E}}\Big). \tag{12}$$

The objective is to derive a joint offloading policy, denoted as $\boldsymbol{\pi}^* = \{\pi_i^{\text{MD}}, \pi_j^{\text{EN}}\}$, that maximizes the long-term cumulative reward for both mobile devices (MDs) and edge nodes (ENs). This objective corresponds to minimizing the overall task delay while optimizing resource utilization across the MEC system. However, solving this optimization problem is particularly challenging due to the need for online and asynchronous decision-making for each task under dynamic and partially observable network conditions involving all MDs and ENs. Formally, the problem is expressed as maximizing the expected discounted cumulative reward as $\mathbb{E}_{\boldsymbol{\pi}^*}\Big[\sum_{i=0}^{\mathcal{I}} \sum_{k=0}^{\infty} \gamma^k \mathbf{r}_{i,k}\Big]$, where $\gamma \in [0,1)$ is the discount factor that determines the importance of future rewards, and $\boldsymbol{\pi}^*$ maps each local observation to a probability distribution over possible actions.

IV. Multi-Agent RL-Based Algorithm

Given the states $\mathbf{o}^{\text{MD}}$ and $\mathbf{o}^{\text{EN}}$, and the actions $\mathbf{a}^{\text{MD}}$ and $\mathbf{a}^{\text{EN}}$, the Q-values for MD $i \in \mathcal{I}$ and EN $j \in \mathcal{J}$ is defined as $Q^{\text{MD}}(\mathbf{o}^{\text{MD}}, \mathbf{a}^{\text{MD}})$ and $Q^{\text{EN}}(\mathbf{o}^{\text{EN}}, \mathbf{a}^{\text{EN}})$, respectively. We introduce an evaluation network to approximate these Q-values as $Q^{\text{MD}}(\mathbf{o}^{\text{MD}}, \mathbf{a}^{\text{MD}}) \approx Q^{\text{MD}}(\mathbf{o}^{\text{MD}}, \mathbf{a}^{\text{MD}}; \theta^{\text{MD}})$ and $Q^{\text{EN}}(\mathbf{o}^{\text{EN}}, \mathbf{a}^{\text{EN}}) \approx Q^{\text{EN}}(\mathbf{o}^{\text{EN}}, \mathbf{a}^{\text{EN}}; \theta^{\text{EN}})$, where weights θ^{MD} and θ^{EN} denote the corresponding network weights. When MD i observes $\mathbf{o}_{i,k}^{\text{MD}}$ or EN j observes $\mathbf{o}_{i,k}^{\text{EN}}$ for task $w_{i,k}$, the offloading agents in MD or EN must choose an action $\mathbf{a}_{i,k}^{\text{MD}}$ and $\mathbf{a}_{i,k}^{\text{EN}}$, respectively, i.e.,

$$\mathbf{a}_{i,k}^{\text{MD}} = \begin{cases} \text{Sample a random action w.p. } \epsilon_i^{\text{MD}}, \\ \arg\max_{\mathbf{a}} Q^{\text{MD}}(\mathbf{o}_{i,k}^{\text{MD}}, \mathbf{a}; \theta_i^{\text{MD}}) \text{ w.p. } 1-\epsilon_i^{\text{MD}}, \end{cases} \tag{13}$$

$$\mathbf{a}_{i,k}^{\text{EN}} = \begin{cases} \text{Sample a random action w.p. } \epsilon_i^{\text{EN}}, \\ \arg\max_{\mathbf{a}} Q^{\text{EN}}(\mathbf{o}_{i,k}^{\text{EN}}, \mathbf{a}; \theta_j^{\text{EN}}) \quad \text{w.p. } 1-\epsilon_j^{\text{EN}}. \end{cases} \tag{14}$$

where ϵ_i^{MD} and ϵ_j^{EN} are the probability of random exploration and decays over time, and θ_i^{MD} and θ_j^{EN} are the weights of of MD i and EN j agents, respectively.

Let a set $\mathcal{B}_i^{\text{MD}}$ and $\mathcal{B}_j^{\text{EN}}$ to denote the replay buffer of MD $i \in \mathcal{I}$ and EN $j \in \mathcal{J}$ to store the experiences. To be specific, the nth and m^{th} experience for MD and EN are represented as $\big(\mathbf{o}^{\text{MD}}(n), \mathbf{a}^{\text{MD}}(n), \mathbf{r}^{\text{MD}}(n), \mathbf{o}^{\text{MD}}(n+1)\big)$ and $\big(\mathbf{o}^{\text{EN}}(m), \mathbf{a}^{\text{EN}}(m), \mathbf{r}^{-}(m), \mathbf{o}^{\text{EN}}(m+1)\big)$, respectively, where $\mathbf{r}^{-}(m) = \mathbf{r}^{\text{MD}}(n)$ is the collabrative reward for both MD and EN involved in case of D2E. Let $\mathcal{D}_i^{\text{MD}}$ and $\mathcal{D}_j^{\text{EN}}$ as batchs of experiences, which are randomly sampled from the replay buffer $\mathcal{B}_i^{\text{MD}}$ and $\mathcal{B}_j^{\text{EN}}$, respectively. Thus, the loss function for updating the weights θ_i^{MD} and θ_j^{EN} agents can be defined as:

$$L\Big(\theta_i^{\text{MD}}\Big) = \frac{1}{|\mathcal{D}_i^{\text{MD}}|} \sum_{n \in \mathcal{D}_i^{\text{MD}}} \Big(Q^{\text{MD}}\big(\mathbf{o}^{\text{MD}}(n), \mathbf{a}^{\text{MD}}(n); \theta_i^{\text{MD}}\big) - \mathcal{U}_n^{\text{MD}} \Big)^2,$$

$$L\Big(\theta_j^{\text{EN}}\Big) = \frac{1}{|\mathcal{D}_j^{\text{EN}}|} \sum_{m \in \mathcal{D}_j^{\text{EN}}} \Big(Q^{\text{EN}}\big(\mathbf{o}^{\text{EN}}(m), \mathbf{a}^{\text{EN}}(m); \theta_j^{\text{EN}}\big) - \mathcal{U}_m^{\text{EN}} \Big)^2.$$

Here $|\mathcal{D}_i^{\text{MD}}|$ and $|\mathcal{D}_j^{\text{EN}}|$ represent the batch sizes for MD $i \in \mathcal{I}$ and EN $j \in \mathcal{J}$, and target values $\mathcal{U}_n^{\text{MD}}$ and $\mathcal{U}_m^{\text{EN}}$ are computed as

$$\mathcal{U}_n^{\text{MD}} = \mathbf{r}(n) + \gamma\, Q^{\text{MD}}\big(\mathbf{o}^{\text{MD}}(n+1), \mathbf{a}_{n+1}; \hat{\theta}_i^{\text{MD}}\big),$$

$$\mathcal{U}_m^{\text{EN}} = \mathbf{r}(m) + \gamma\, Q^{\text{EN}}\big(\mathbf{o}^{\text{EN}}(m+1), \mathbf{a}_{m+1}; \hat{\theta}_j^{\text{EN}}\big),$$

where $\mathbf{a}_{n+1} = \arg\max_{\mathbf{a}} Q(\mathbf{o}^{\text{MD}}(n+1), \mathbf{a}; \theta_i^{\text{MD}})$ and $\mathbf{a}_{m+1} = \arg\max_{\mathbf{a}} Q(\mathbf{o}^{\text{EN}}(m+1), \mathbf{a}; \theta_i^{\text{EN}})$, with the parameters of the corresponding target networks $\hat{\theta}_i^{\text{MD}}$ and $\hat{\theta}_j^{\text{EN}}$ for the MD $i \in \mathcal{I}$ and EN $j \in \mathcal{J}$, respectively.

References

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug 2019.

[2] C. Jiang, Z. Luo, L. Gao, and J. Li, "A truthful incentive mechanism for movement-aware task offloading in crowdsourced mobile edge computing systems," *IEEE Internet of Things Journal*, vol. 11, no. 10, pp. 18 292–18 305, 2024.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb 2015.

[4] M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[5] N. Zhao, Y.-C. Liang, D. Niyato, Y. Pei, M. Wu, and Y. Jiang, "Deep reinforcement learning for user association and resource allocation in heterogeneous cellular networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 11, pp. 5141–5152, Nov 2019.

[6] M. Tang and V. W. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans Mob Comput*, vol. 21, no. 6, pp. 1985–1997, Nov 2020.

[7] H. Zhou, K. Jiang, X. Liu, X. Li, and V. C. Leung, "Deep reinforcement learning for energy-efficient computation offloading in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 1517–1530, Jun 2021.

[8] L. Liao, Y. Lai, F. Yang, and W. Zeng, "Online computation offloading with double reinforcement learning algorithm in mobile edge computing," *J. Parallel Distrib. Comput.*, vol. 171, pp. 28–39, Jan 2023.

[9] C. Sun, X. Li, C. Wang, Q. He, X. Wang, and V. C. Leung, "Hierarchical deep reinforcement learning for joint service caching and computation offloading in mobile edge-cloud computing," *accepted for publication in IEEE Trans. Services Computing*, Aug 2024.

[10] I. Rahmaty, H. Shah-Mansouri, and A. Movaghar, "QECO: A QoE-oriented computation offloading algorithm based on deep reinforcement learning for mobile edge computing," *arXiv preprint arXiv:2311.02525*, 2024.

[11] P. Hernandez-Leal, M. Kaisers, T. Baarslag, and E. M. De Cote, "A survey of learning in multiagent environments: Dealing with non-stationarity," *arXiv preprint arXiv:1707.09183*, 2017.

[12] J. Yang, Q. Yuan, S. Chen, H. He, X. Jiang, and X. Tan, "Cooperative task offloading for mobile edge computing based on multi-agent deep reinforcement learning," *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 3205–3219, 2023.

[13] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Cooperative task offloading and block mining in blockchain-based edge computing with multi-agent deep reinforcement learning," *IEEE Transactions on Mobile Computing*, vol. 22, no. 4, Apr 2021.

[14] H. Gao, X. Wang, W. Wei, A. Al-Dulaimi, and Y. Xu, "Com-ddpg: Task offloading based on multiagent reinforcement learning for information-communication-enhanced mobile edge computing in the internet of vehicles," *IEEE Transactions on Vehicular Technology*, 2023.

[15] S. Tan, B. Chen, D. Liu, J. Zhang, and L. Hanzo, "Communication-assisted multi-agent reinforcement learning improves task-offloading in uav-aided edge-computing networks," *IEEE Wireless Commun. Letters*, vol. 12, no. 12, pp. 2233–2237, Des 2023.

[16] F. A. Oliehoek, C. Amato *et al.*, *A concise introduction to decentralized POMDPs*. Springer, 2016, vol. 1.

[17] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. of AAAI Conference on Artificial Intelligence*, Phoenix, AZ, Feb 2016.

[18] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. of International Conference on Machine Learning*. New York, NY, Jun 2016.